

3. [Obligatorio] Escribir el algoritmo planteado. Describir y justificar la complejidad de dicho algoritmo.

El algoritmo consta de dos etapas principales:

1. Ordenamiento de los rivales:

Se insertan los rivales uno a uno en una lista, manteniendo el orden decreciente según el valor a_i , utilizando la función :

```
def ordenar_rivales(arr_rivales, rival):
    i = 0
    while i < len(arr_rivales) and arr_rivales[i][1] >= rival[1]:
        i += 1
    arr_rivales.insert(i, rival)
```

Esta función recorre la lista hasta encontrar la posición correcta para insertar el nuevo rival.

La carga de los datos se realiza mediante:

```
def formatear_archivo_entrada():
    nombre_archivo = sys.argv[1]
    rivales = []

    with open(nombre_archivo, "r") as f:
        next(f)

        for linea in f:
            linea = linea.strip()
            if linea == "":
                continue

            s, a = map(int, linea.split(","))
            rival = [s, a]

            ordenar_rivales(rivales, rival)

    return rivales
```

2. Cálculo del tiempo total}

El cálculo del tiempo total se realiza mediante un único recorrido de la lista de rivales ya ordenada en la función:

```
def analizar_rivales(rivales):
    tiempo_scaloni = 0
    tiempo_total = 0

    for s, a in rivales:
        tiempo_scaloni += s
        fin_rival = tiempo_scaloni + a

    if fin_rival > tiempo_total:
        tiempo_total = fin_rival
```

return tiempo_total

En cada iteración se acumula el tiempo de Scaloni y se calcula el tiempo de finalización del rival actual como la suma del tiempo acumulado más el tiempo del ayudante.

Siendo el resultado final ,el máximo de estos tiempos de finalización.

Complejidad:

Sea n la cantidad de rivales.El costo dominante del algoritmo está en la función de inserción:

```
while i < len(arr_rivales) and arr_rivales[i][1] >= rival[1]:
```

```
    i += 1
```

Esta estructura implica que, para cada inserción, se puede recorrer toda la lista.

explicación matematica:

Sea $T(n)$ el tiempo total de ejecutar el algoritmo para n rivales.

Para cada rival k, el costo de insertarlo es proporcional al tamaño actual de la lista:

$$costo(k) = k - 1$$

Por lo tanto:

$$T(n) = \sum_{k=1}^n (k - 1) = \sum_{k=1}^n k - \sum_{k=1}^n 1 \text{ sabemos que: } \sum_{k=1}^n k = \frac{n(n+1)}{2} \text{ y } \sum_{k=1}^n 1 = n \text{ entonces:}$$

$$T(n) = \frac{n(n+1)}{2} - n \text{ simplificamos la ecuación: } T(n) = \frac{n^2+n-2n}{2} = \frac{n^2-n}{2} \text{ para } n \text{ grande: } T(n) \in O(n^2)$$

El siguiente pedazo de código realiza un recorrido de la lista, por lo tanto su complejidad seria $T(n) = O(n)$

for s, a in rivales:

COMPLEJIDAD TOTAL:

sumamos ambos costos de codigo:

$$T(n) = O(n^2) + O(n) \text{ dado que el término dominante es cuadrático: } T(n) = O(n^2)$$

explicación escrita :

El algoritmo presenta una complejidad cuadrática debido al proceso de inserción ordenada, que implica recorrer la lista parcialmente construida en cada iteración.

El cálculo del tiempo total es lineal y no afecta el orden dominante.

Dado que el algoritmo no presenta una estructura recursiva, el Teorema Maestro no resulta aplicable. En consecuencia, la complejidad total del algoritmo es $O(n^2)$.

5. [Opcional] Analizar si (y cómo) afecta la variabilidad de los valores de a_i y s_i a la optimalidad del algoritmo planteado.

El algoritmo planteado se basa en ordenar los rivales de forma decreciente según a_i , es decir, priorizar aquellos que más tiempo le llevan a los ayudantes. La idea detrás de esto es que los análisis más largos comiencen lo antes posible, aprovechando el paralelismo.

Es importante notar que este criterio depende del **orden relativo de los valores de a_i** y no de sus valores absolutos. Por lo tanto, aunque los valores cambien mucho (sean grandes, chicos o similares), mientras se mantenga el orden entre ellos, la solución sigue siendo óptima.

Para analizar esto, consideramos distintos casos:

- **Caso 1: $a_i \gg s_i$**

En este escenario, los tiempos de los ayudantes dominan el resultado. Si no se ordenan correctamente (de mayor a menor), el mayor a_i podría quedar al final, aumentando el tiempo total. El algoritmo greedy evita esto, por lo que sigue siendo óptimo.

- **Caso 2: $a_i \approx s_i$**

Cuando los valores son similares, tanto Scaloni como los ayudantes influyen en el resultado. Sin embargo, el criterio greedy sigue siendo válido, ya que cualquier cambio que viole el orden de a_i puede empeorar el resultado.

- **Caso 3: $s_i \gg a_i$**

En este caso, el tiempo total está dominado por la suma de los s_i . Aun así, el orden según a_i sigue siendo el correcto, ya que cambiarlo no garantiza ninguna mejora y puede generar soluciones peores.

- **Caso 4: $s_i = a_i$**

Aquí ambos valores tienen el mismo peso. El algoritmo sigue funcionando correctamente, y cualquier orden que respete el criterio greedy mantiene la optimalidad.

En conclusión, la variabilidad de los valores de s_i y a_i puede afectar el **valor final del tiempo total**, pero no afecta la **optimalidad del algoritmo**. Esto se debe a que la

correctitud del enfoque greedy está basada en el orden de los a_i , lo cual se mantiene independientemente de cómo varíen los datos.